

Informe de Auditoría del Sistema Web

Integrantes del grupo:

Jorge Illera Rivera

Iker Argulo Galán

Oscar Lijerón Pérez

Bruno Izaguirre Martínez de Marañón

María Bogajo

Ander Javier Corral

Fecha:

09/11/2024

Índice de Contenidos

1	Introducción	3
2	Metodología	3
3	Vulnerabilidades	3
3.1	Rotura de control de acceso	3
3.1.1	Soluciones implementadas	3
3.2	Fallos criptográficos	4
3.2.1	Soluciones implementadas	4
3.3	Inyección	5
3.3.1	Cross-Site Scripting (XSS)	6
3.3.2	Inyección SQL	6
3.3.3	Soluciones implementadas	7
3.4	Diseño inseguro	8
3.4.1	Soluciones implementadas	8
3.5	Configuración de seguridad insuficiente	8
3.5.1	Credenciales por defecto	9
3.5.2	Soluciones implementadas	9
3.6	Componentes vulnerables y obsoletos	9
3.6.1	Soluciones implementadas	9
3.7	Fallos de identificación y autenticación	9
3.7.1	Soluciones implementadas	10
3.8	Fallos en la integridad de datos y software	10
3.8.1	Soluciones implementadas	10
3.9	Fallos en la monitorización de la seguridad	10
3.9.1	Soluciones implementadas	10
4	Conclusiones finales	11
5	Bibliografía	11
6	Apéndices	12

Índice de Figuras

1	Errores de inyección detectados por ZAP.	5
2	Desplazamiento del botón Juegos.	8

Índice de Tablas

1 Introducción

Este informe detalla la auditoría de seguridad del sistema web utilizando OWASP ZAP, con el objetivo de identificar y mitigar vulnerabilidades.

2 Metodología

La auditoría se realizó mediante un análisis de ZAP, ejecutando...

3 Vulnerabilidades

3.1 Rotura de control de acceso

Una rotura del control de acceso ocurre cuando un sistema permite a usuarios acceder a datos, funciones o recursos para los que no tienen autorización, debido a fallos en la implementación de los permisos. Esto puede resultar en accesos no autorizados, modificaciones indebidas de datos o acciones privilegiadas que deberían estar restringidas.

3.1.1 Soluciones implementadas

Para minimizar la utilización de CORS¹, se ha integrado el uso de tokens CSRF² con el fin de implementar mecanismos de acceso únicos.

- **Integración de tokens CSRF**

Se ha implementado la creación de los tokens en archivos donde se manejan formularios o se ingresan los identificadores y contraseñas de los usuarios. Esto incluye archivos como `login.php`, `register.php`, `add.item.php` y `modify.user.php`.

1. **Generación del token CSRF**

Si no existe ya un token CSRF en la sesión `$_SESSION['csrf_token']`, se genera uno nuevo usando `bin2hex(random_bytes(32))`. Con esto se asegura que cada sesión tiene un token único y difícil de adivinar.

2. **Uso del token en el formulario**

En el formulario HTML³ (HyperText Markup Language)(`login.html`), este token se debe enviar como un campo oculto. Esto permite que el servidor valide que el formulario fue enviado desde la misma aplicación.

¹CORS: Mecanismo que permite que los recursos de un servidor sean accesibles desde diferentes dominios de manera controlada.

²CSRF: Códigos únicos generados para verificar que una solicitud proviene de una fuente legítima y no de un atacante.

³Lenguaje de marcado utilizado para estructurar y organizar el contenido en las páginas web.

3. Validación del token en el servidor

Cuando el formulario se envía, el código verifica si el token recibido (`$_POST['csrf_token']`) coincide con el que está almacenado en la sesión. Si no coinciden o el token no existe, la solicitud se considera no válida y se bloquea (`die("Error de validación CSRF");`).

4. Eliminación del token

Una vez que el usuario inicia sesión correctamente, el token se elimina usando el comando `unset($_SESSION['csrf_token'])`. Esto asegura que no pueda ser reutilizado.

3.2 Fallos criptográficos

En esta sección se comentarán los fallos de seguridad web relacionados con la criptografía⁴ (o la falta de esta) que causan la exposición de datos sensibles[9].

Se han identificado y solucionado dos fallos criptográficos. El primer fallo consiste en almacenar la información de la base de datos sin cifrar, utilizando únicamente el cifrado de datos en reposo incorporado por nuestra base de datos, el cual se descifra directamente al realizar una consulta[12]. El segundo fallo detectado es almacenar las contraseñas en texto plano.

3.2.1 Soluciones implementadas

Para subsanar el primer error se ha decidido cifrar la información que se almacena en la base de datos. El algoritmo de cifrado simétrico⁵ que se ha usado es el AES (Advanced Encryption Standar) de 256 bits de longitud, puesto que este algoritmo es considerado seguro a día de hoy[11].

El uso de este algoritmo requiere una clave segura para cifrar y descifrar los datos. Para ello se ha usado una función[6] de Python⁶ que genera la clave usando la librería `secrets`, que es una librería que genera números aleatorios con la suficiente entropía para que la clave generada pueda ser considerada criptográficamente fuerte[5]. Una vez generada la clave de secreta se guarda como una variable de entorno⁷ en un archivo `.env` llamado `claveCifrado.env` para evitar comprometerla. Los datos cifrados de la tabla Juegos son el nombre, género, fecha de lanzamiento, valoración y precio. Mientras que los datos cifrados de la tabla Usuarios son el nombre, teléfono, DNI y fecha de nacimiento. Sin embargo, como al cifrar los datos su número de caracteres aumenta, se han tenido que cambiar los tipos de dato y la longitud de los campos de la base de datos para poder almacenarlos.

⁴Práctica que consiste en proteger información mediante el uso de algoritmos codificados, Hashes y firmas.

⁵Método de cifrado que usa la misma clave para cifrar y descifrar.

⁶Lenguaje de programación orientado a objetos usado principalmente para el desarrollo web y de aplicaciones informáticas.

⁷Las variables de entorno son variables disponibles para tu programa/aplicación de forma dinámica durante el tiempo de ejecución. El valor de estas variables puede proceder de diversas fuentes: archivos de texto, gestores secretos de terceros, scripts de llamada, etc.[7]

Para solventar el segundo error se ha decidido usar un algoritmo resumen para generar el resumen criptográfico o Hash⁸ de la contraseña en vez de guardarla como texto plano. Para ello se ha usado la función `password_hash` a la que se le pasa como parámetros la contraseña a resumir y una constante de PHP⁹ (Hypertext Preprocessor) `PASSWORD_BCRYPT` que indica que queremos usar el algoritmo Bcrypt[8]. Bcrypt usa sales¹⁰ para mayor seguridad ya que hace que dos contraseñas iguales de usuarios diferentes generen resúmenes criptográficos distintos, por lo que se hace mucho mas difícil precalcularlos.

3.3 Inyección

¿Qué es la inyección?

La inyección es una vulnerabilidad de seguridad en la que un atacante introduce datos maliciosos en una aplicación, a menudo mediante campos de entrada o URL, logrando manipular comandos o consultas dentro del sistema. Esto permite ejecutar código no autorizado, acceder a datos sensibles o modificar información sin permisos[1].

En esta sección se han identificado dos amenaza de alto riesgo. Las vulnerabilidades detectadas corresponden a una Inyección SQL y a un ataque de Cross-Site Scripting (XSS) Reflejado, como se puede observar en la Figura 1.



Figura 1: Errores de inyección detectados por ZAP.

⁸Criptograma que consiste en una sucesión de caracteres alfanuméricos que representan la totalidad del contenido original.

⁹PHP es un lenguaje de código abierto especialmente adecuado para el desarrollo web y que puede ser incrustado en HTML.[4]

¹⁰Caracteres aleatorios que se le añaden a la contraseña para que sea mas difícil precalcular los resúmenes criptográficos.

3.3.1 Cross-Site Scripting (XSS)

La vulnerabilidad CWE-79 corresponde a Cross-Site Scripting (XSS) Reflejado, que permite a un atacante inyectar código malicioso en los parámetros de entrada del sistema, como los campos de formularios o las URL. Cuando un usuario accede a una URL o un formulario que contiene este código inyectado, el navegador del usuario ejecuta dicho código, lo que puede llevar a la manipulación de datos, el robo de cookies, la ejecución de scripts no autorizados, y otros ataques de seguridad.[2]

Causa de la vulnerabilidad

La vulnerabilidad XSS detectada se originaba en el archivo `modify_item.php`, más concretamente en el campo de la fecha de lanzamiento y en otros campos de entrada en el formulario de modificación de un juego. Esto se debía a que los valores de entrada eran reflejados en la respuesta sin la sanitización o validación adecuada, permitiendo a los atacantes inyectar y ejecutar código JavaScript en el navegador del usuario.

Pruebas y comportamiento observado

Sin la sanitización de entrada: OWASP ZAP detectó que ciertos campos permitían la inyección de código JavaScript en los valores de los formularios.

Por ejemplo, en el campo de la fecha, se podía ingresar un valor como

`<script>alert(1);</script>`, lo cual provocaba la ejecución de una alerta en el navegador.

3.3.2 Inyección SQL

La vulnerabilidad CWE-89 corresponde a la inyección SQL, donde un atacante es capaz de insertar o manipular consultas SQL a través de los datos de entrada, en este caso, el parámetro `id` en la URL.

Causa de la Vulnerabilidad

La vulnerabilidad detectada se originaba en el archivo `show_user.php` y fue causada por la construcción dinámica de consultas SQL sin la adecuada sanitización¹¹ ni parametrización de los datos proporcionados por el usuario. En este caso, el valor del parámetro `id` se concatenó directamente a la consulta SQL sin protección, lo que permitió la ejecución de código SQL malicioso.

Pruebas y comportamiento observado

En el error encontrado, la inyección fue posible al modificar el parámetro de la siguiente manera: `1750' AND '1'='1' --`

Esto permitió a OWASP ZAP modificar la consulta SQL ejecutada en el servidor, manipulando los resultados devueltos y accediendo a información que no debía ser revelada. En particular, se manipuló la consulta para obtener datos utilizando una condición booleana que siempre era verdadera, `'1'='1'`, y luego se comentó el resto de la consulta utilizando `--`, lo que invalidaba el resto de la consulta SQL.

¹¹Sanitización se refiere al proceso de limpiar o filtrar los datos de entrada para eliminar cualquier contenido malicioso que pudiera comprometer la seguridad del sistema.[3]

3.3.3 Soluciones implementadas

Para mitigar la vulnerabilidad de XSS se implementaron las siguientes soluciones:

- **Escapado¹² de salida:**

Se utilizó la función `htmlspecialchars()` en PHP para escapar caracteres especiales (como `¡`, `¿`, `&`, `"`, `'`) en todas las salidas. Esto asegura que el contenido reflejado en el navegador sea tratado como texto y no como código ejecutable.

- **Validación y sanitización de fecha de lanzamiento:**

Se implementó una expresión regular (`preg_match`) para asegurarse de que las fechas ingresadas cumplan con el formato `YYYY-MM-DD`.

- **Sanitización de datos entrantes:**

Para cada valor de entrada recibido a través del formulario, se aplicó `mysqli_real_escape_string()` para evitar que cualquier código especial o malicioso llegue a la base de datos.

- **Validación de Campos Numéricos (rating y price):**

Los valores de `rating` y `price` se convirtieron a tipo `float` usando `floatval()` para asegurarse de que se almacenen y procesan correctamente como valores numéricos.

Para mitigar la vulnerabilidad de Inyección SQL se implementaron las siguientes soluciones:

- **Uso de consultas preparadas:**

Se ha utilizado el enfoque de consultas preparadas con declaraciones `bind_param()`. Esto asegura que los parámetros sean tratados de forma segura, separando los datos del código SQL, y evitando así que cualquier entrada maliciosa pueda modificar la estructura de las consultas.

- **Validación y sanitización de entradas**

Además de las consultas preparadas, se valida y filtra las entradas del usuario para asegurarse de que solo se acepten datos válidos. Por ejemplo, el uso de `intval()` para el parámetro `id` convierte cualquier valor no numérico a un valor seguro (como 0), evitando que se inyecten datos inesperados en las consultas.

Todas estas soluciones también se han implementado en otros archivos que, aunque no presentaban errores en el análisis, tenían vulnerabilidades relacionadas con la inyección que necesitaban ser abordadas.

Resultados de la segunda auditoría

Después de implementar las soluciones descritas, se realizó un segundo análisis con OWASP ZAP. Los resultados fueron positivos: no se detectaron vulnerabilidades de XSS ni otros problemas de inyección.

Conclusiones

Ambas vulnerabilidades fueron una amenaza crítica para la aplicación, ya que podía permitir a los atacantes ejecutar código malicioso en el navegador de los usuarios o insertar consultas SQL

¹²Escapado se refiere al proceso de transformar caracteres especiales en una entrada válida para que no sean interpretados como parte del código ejecutable.[3]

a través de los datos de entrada. Gracias a las soluciones implementadas, las vulnerabilidades se han mitigado de manera efectiva. La aplicación ahora cumple con un estándar de seguridad más alto, protegiendo los datos frente a posibles ataques de inyección de código.

3.4 Diseño inseguro

¿Que significa diseño inseguro?

Una página web tiene un diseño inseguro cuando su estructura, funcionalidades, o configuración comprometen la seguridad de los usuarios o de sus datos. Esto puede deberse a malas prácticas en el desarrollo, falta de medidas de seguridad adecuadas o errores en la implementación. Además, se ha desplazado la posibilidad de ver, modificar y añadir videojuegos a la plataforma, exclusivamente para usuarios registrados.

3.4.1 Soluciones implementadas

Para mitigar las vulnerabilidades de ???la no validación de datos???, se han implementado las siguientes soluciones:

Desplazamiento del botón para ver/añadir/modificar juegos:

Se ha desplazado el botón de la página inicial a una página posterior después de haber iniciado sesión, como se puede apreciar en la Figura 2. Con esto se consigue evitar que usuarios sin haberse registrado e iniciado sesión puedan llegar a saturar la base de datos y que esta deje de responder.



Figura 2: Desplazamiento del botón Juegos.

3.5 Configuración de seguridad insuficiente

¿Qué es la configuración de seguridad insuficiente?

La configuración de seguridad insuficiente ocurre cuando un sistema o aplicación utiliza configuraciones predeterminadas, credenciales simples o no establece medidas de protección adecuadas contra posibles amenazas. Esto puede incluir la falta de restricciones de acceso, configuraciones

inadecuadas de servicios, y uso de contraseñas débiles o predeterminadas [13]. En esta sección se ha detectado una vulnerabilidad principal, el uso de credenciales (usuario y contraseña) por defecto o extremadamente simples.

3.5.1 Credenciales por defecto

Esta vulnerabilidad se veía reflejada en dos de nuestros servicios, MariaDB y phpMyAdmin.

Credenciales de MariaDB:

Usuario: admin

Contraseña: test

Al igual que en phpMyAdmin:

Usuario: admin

Contraseña: test

Estas credenciales son extremadamente simples y no cumplen con las políticas básicas de seguridad para la autenticación. Y además, dejan expuestos servicios críticos, aumentando significativamente el riesgo de ataques como la inyección SQL, robo de datos y escalamiento de privilegios.

3.5.2 Soluciones implementadas

Para mitigar este problema, se han modificado tanto los usuarios como las contraseñas de ambos servicios por unos valores más seguros. Las nuevas contraseñas cuentan con más de 10 dígitos entre los que se incluyen combinaciones de mayúsculas, minúsculas, números y caracteres especiales.

Conclusiones

El uso de credenciales predeterminadas o débiles representaba un riesgo crítico para la seguridad del sistema, ya que las anteriores eran especialmente conocidas y fáciles de explotar por atacantes. Esto facilitaba y posibilitaba el acceso no autorizado a servicios clave como MariaDB y phpMyAdmin, comprometiendo la integridad y confidencialidad de los datos.

3.6 Componentes vulnerables y obsoletos

3.6.1 Soluciones implementadas

3.7 Fallos de identificación y autenticación

¿Qué es un fallo de identificación y autenticación?

Un fallo de identificación ocurre cuando el sistema no reconoce quién dices ser. Esto puede deberse a varias razones, como ingresar datos incorrectos o inexistentes, que el sistema no busque correctamente el usuario en la base de datos, o incluso por el uso de mensajes de error ambiguos o confusos que no ayudan al usuario a identificar el problema. Además, la ausencia de mensajes claros también puede contribuir a que el sistema no reconozca correctamente al usuario.

Esto puede suceder porque el usuario ha registrado una contraseña insegura o fácil de adivinar,

utiliza mecanismos simples para recuperar contraseñas, o bien porque no se ha implementado un sistema de autenticación más robusto como la autenticación multifactor¹³.

¿Cómo ocurre un ataque por fallos de identificación o autenticación?

Esta vulnerabilidad se da cuando un atacante obtiene una lista de usuarios y contraseñas que pueden ser válidas en el sistema, y luego intenta acceder a las cuentas mediante un proceso automatizado. Un ejemplo común es el ataque de Credential stuffing¹⁴. En este ataque, el atacante utiliza credenciales filtradas de otros servicios donde los usuarios han utilizado las mismas contraseñas, y las prueba en el sistema objetivo para obtener acceso a sus cuentas. Si el sistema no implementa medidas adicionales de seguridad, como la autenticación multifactor o límites de intentos de acceso fallidos, el atacante puede obtener acceso a las cuentas de usuario con relativa facilidad.

3.7.1 Soluciones implementadas

Para reducir los riesgos de los errores en la identificación y autenticación se ha implementado la siguientes soluciones:

- **Registro de todos los intentos de sesiones:**

Se ha utilizado un archivo .log¹⁵ para poder gestionar y observar el por qué de los errores de dichos intentos de sesiones. En el archivo .log se registra lo siguiente:

1. Fecha y hora en formato yyyy-mm-dd hh:mm:ss.
2. SUCCESS o FAILED.
3. Email registrado o no en la base de datos.
4. IP¹⁶.

3.8 Fallos en la integridad de datos y software

3.8.1 Soluciones implementadas

3.9 Fallos en la monitorización de la seguridad

3.9.1 Soluciones implementadas

Bloqueo temporal de usuarios por Logins¹⁷ fallidos:

Con ayuda del archivo .log se consigue registrar aquellos usuarios que han fallado 5 veces

¹³Autenticación multifactor se refiere a un método de autenticación que requiere que el usuario proporcione dos o más factores de verificación diferentes para acceder a un sistema

¹⁴Credential stuffing: Ataque cibernético en el que los atacantes usan combinaciones de nombres de usuario y contraseñas robadas de otras plataformas para intentar acceder a cuentas en diferentes sitios web o servicios.

¹⁵Un .log es un archivo de texto que se utiliza para registrar eventos, actividades o mensajes de un sistema, programa o aplicación

¹⁶IP: Identificador único que se asigna a cada dispositivo conectado a una red

¹⁷Proceso mediante el cual un usuario se identifica y autentica en un sistema, servicio o aplicación.

el inicio de sesión en los últimos 10 minutos. Dicha cuenta se bloquea por 10 minutos y si el usuario intenta hacer nuevamente un Login, se le mostrará el tiempo que le falta para que pueda volver a realizar otros 5 intentos. Esto se logra gestionar en el archivo.log con la función: `function checkFailedAttempts(email, log_file, max_attempts = 5, lock_time = 600)`

4 Conclusiones finales

5 Bibliografía

References

- [1] owasp.org
Injection
<https://owasp.org/www-community/Injection> (accedido 11/07/2024).
- [2] owasp.org
Cross-Site Scripting (XSS)
<https://owasp.org/www-community/attacks/xss/> (accedido 11/07/2024).
- [3] cwe.mitre.org
Sanitización y escapeo
<https://owasp.org/www-community/attacks/xss/> (accedido 11/07/2024).
- [4] php.net
¿Qué es PHP?
<https://www.php.net/manual/es/intro-what-is.php> (accedido 11/07/2024).
- [5] python.org
¿Qué es la librería secrets?
<https://docs.python.org/es/3/library/secrets.html> (accedido 21/11/2024).
- [6] Python Code: Generación de Claves de Cifrado
Este código en Python se utiliza para generar claves criptográficamente seguras usando la librería `secrets` basado en código generado por ChatGPT.

```
import secrets

def generar_clave_encriptacion(longitud=32):

    clave = secrets.token_bytes(longitud)    # Devuelve una cadena
        de bytes aleatorios

    return clave.hex()
```

```
clave_encriptacion = generar_clave_encriptacion()

print(f"Clave de encriptacion generada: {clave_encriptacion}")
}
```

[7] kinsta.com

¿Que es una variable de entorno?

<https://kinsta.com/es/base-de-conocimiento/variables-de-entorno/> (accedido 15/11/2024).

[8] skysnag.com

¿Que es Bcrypt?

<https://www.skysnag.com/es/blog/what-is-bcrypt/> (accedido 15/11/2024).

[9] owasp.org

¿Que son los fallos criptográficos?

https://owasp.org/Top10/es/A02_2021-Cryptographic_Failures/ (accedido 16/11/2024).

[10] aws.amazon.com

¿Que es la criptografía?

<https://aws.amazon.com/es/what-is/cryptography/> (accedido 16/11/2024).

[11] msmk.university

¿Qué es AES? ¿Es seguro?

<https://msmk.university/advanced-encryption-standard-aes/> (accedido 16/11/2024).

[12] mariadb-com

Limitaciones e información sobre el cifrado de MariaDB

https://mariadb-com.translate.google.com/translate?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=rq (accedido 21/11/2024).

[13] owasp.org

Configuración de seguridad insuficiente

https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ (accedido 22/11/2024).

6 Apéndices